

Ansible Automation Platform 2.7

Automation Portal and Content Lifecycle

Version 1.0.0, 2026-09-22

Home

Introduction

The Ansible Automation Portal with Ansible Automation Platform (AAP) serves as a unified interface that shifts the platform focus from simple execution to a comprehensive "build, discover, and run" experience. It simplifies enterprise automation for users of all skill levels by replacing complex CLI workflows with a streamlined, point-and-click web experience.

Duration: 1 hours

Topics covered in this course

- Automation Portal

Prerequisites

This course assumes that you have the following prior experience:

- Basic understanding of Ansible Automation Platform and OpenShift Container Platform (OCP).

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Amrinder Singh (Content Architect)
- Anna Swicegood (Learning Product Manager)

Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. Log in to the [RHDP portal](#).
2. Search for the catalog entry: **Advanced Deployment of Ansible Automation Platform 2.7**.



Product Enablement: Advanced Deployment of AAP 2.7

provided by Product Technical Learning

Order



 Save as favorite

Category
Labs

Provider
Product Technical Learning

Product Family
Red Hat Automation

Rating
★★★★☆ (15)

Cost
\$0.16/hr \$\$\$

Estimated provision time
±24 minutes

Uptime 
 100%

Last update
3 hours ago

Last successful provision
29 hours ago

Auto-Stop
6 Hours

Auto-Destroy
48 Hours

Description

Learn about AAP 2.7 advanced deployment topologies, Automation Mesh sizing, and hands-on containerized and operator-based installation across multiple nodes.

Ansible Automation Platform 2.7 enables scalable IT automation across complex environments. This lab pre-installs the AAP operator on OpenShift and provisions three dedicated VMs – an execution node, a containerized install host, and a container execution node – so you can practice real-world multi-node deployment patterns.

Lab Guide

You can access the lab guide though.

- [Red Hat LMS](#) (preferred)
- [Quick Course](#)

What Content Is Covered In The Lab?

- Understanding of Automation Mesh and multi-node topologies
- Calculate the sizing requirement of Ansible Automation Platform
- Operator-based AAP deployment on OpenShift with Automation Mesh.
- Containerized AAP deployment across dedicated VMs with Automation Mesh.
- Troubleshooting Ansible Automation Platform installation issues

Environment Specifications

- **OpenShift SNO Cluster** – Pre-installed AAP operator

3. Click on the catalog entry in the search results.
4. On the catalog page, click the **Order** button.
5. Fill out the required details in the order form.

Activity *

- ☐ Customer Facing
- ☐ Partner Facing
- ☒ Practice / Enablement

Purpose *

Trying out a technical solution ▼



Salesforce ID *(Opportunity ID, Campaign ID, CDH Party or Project ID)*

- ☐ Campaign ☐ CDH ☐ Opportunity ☐ Project 

Salesforce ID

 Search



☐ I'll provide the Salesforce ID within 48 hours. 

6. **Review the warning** at the bottom of the form and **check the box** labeled:
“I confirm that I understand the above warnings.”
7. **Click the Order** button to place your lab order.

Important Notes:

- This lab may take approximately **20 minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: <https://demo.redhat.com/>
- RedHat partners: <https://partner.demo.redhat.com/>

Automation Portal

The Ansible Automation Portal with Ansible Automation Platform (AAP) serves as a unified interface that shifts the platform focus from simple execution to a comprehensive "build, discover, and run" experience. It simplifies enterprise automation for users of all skill levels by replacing complex CLI workflows with a streamlined, point-and-click web experience.

Key features include:

- **Simplified User Experience:** Designed for business users, reducing the need for deep technical knowledge of Ansible playbooks.
- **Platform Integration:** Deployed on OpenShift Container Platform (OCP) with support for RBAC-based navigation, allowing administrators to reduce UI complexity through role-based access policies.
- **Role-Based Access Control:** Facilitates user role separation, ensuring that users only interact with relevant automation assets.
- **Unified Workflow:** Acts as a centralized hub for managing the full lifecycle of automation, from building Execution Environments to discovering and running pre-defined automation content.

Automation Portal: Installation Overview with Architectural Benefits

The Automation Portal with Ansible Automation Platform provides a centralized interface for managing automation workflows, visual environment building, and content discovery. When deploying on OpenShift Container Platform (OCP), the Automation Portal leverages the Kubernetes-native architecture to ensure scalability, high availability, and seamless integration with existing AAP controller components.

The Evolution of the Portal

In previous iterations, such as AAP 2.6/7, the portal functioned primarily as an execution layer. Users navigated to the interface, identified authorized job templates, populated surveys, and triggered automation.

Automation Portal 2.7 expands this scope to support three primary pillars: * **Visual Execution Environment (EE) Builder:** A GUI-driven workflow for container image creation. * **Centralized Content Discovery:** A unified catalog that aggregates automation assets from multiple sources. * **Platform-wide Governance:** Enhanced RBAC controls, administrative navigation policies, and AI-assisted operational workflows.

Architectural Benefits

The shift toward this "Build, Discover, and Run" architecture provides several key advantages for organizations:

1. Unified Developer Productivity

By integrating the EE Builder and the Content Discovery catalog directly into the portal, developers no longer need to switch context between the command line, CI/CD pipelines, and the execution interface. This reduces the "friction of automation," allowing teams to define, build, and deploy automation assets without leaving the AAP ecosystem.

2. Reduced Complexity via Admin-Defined RBAC

The architectural design of the 2.7 portal emphasizes role-based navigation. Administrators can configure custom menus to match user roles: * **End-Users:** May see a simplified view restricted to "Templates" and "Job History," minimizing cognitive load and accidental configuration changes. * **Platform Engineers:** Retain access to the full portal, including builder tools, catalog management, and administrative policy settings.

3. Streamlined Governance and Discovery

The architecture supports multi-source aggregation. By centralizing content from various repositories (GitHub, GitLab, Private Automation Hub), the portal provides a "browse-before-build" workflow. This allows teams to trace existing automation assets, preventing the duplication of effort and ensuring that teams are using approved, organization-wide standards.

4. Extensibility and AI Readiness

With the inclusion of an MCP (Model Context Protocol) server, the portal is architected to support AI-assisted operations. This allows for natural language interaction with the platform, enabling users to query automation status or trigger workflows through supported AI clients, further lowering the barrier to entry for non-technical stakeholders.

By consolidating these functions, Automation portal reduces the operational overhead associated with maintaining disparate build tools and content repositories, ensuring that automation scaling remains consistent with enterprise security and governance policies.

Prerequisites and Requirements

Before initiating the deployment, ensure your environment meets the following technical requirements:

- **Platform Subscription:** Valid Red Hat Ansible Automation Platform subscription.
- **Existing AAP Infrastructure:** An active instance of AAP 2.6+ to facilitate authentication and integration.
- **OpenShift Environment:** Access to an OpenShift Container Platform cluster (version 4.12–4.17+).
- **Permissions:** Cluster-level permissions to create OAuth applications within the target project.
- **CLI Tools:**
 - **oc** CLI tool for OpenShift management.
 - **helm** (version 3.10 or newer) for package deployment.

Installation Overview

The official deployment method uses **Helm charts** on OpenShift Container Platform. The Helm chart automates the deployment of the Automation Portal, handling networking, storage, and integration with your AAP instance.



Two-Phase Installation Process:

1. **Phase 1 - Prerequisites:** Complete set up OAuth, API tokens, OC cli and OpenShift secrets configuration.
2. **Phase 2 - Deployment:** Deploy the Automation Portal via Helm chart, configure the necessary environment variables, and validate the installation.

Both phases are required for a successful installation.

Managing RBAC-based Navigation Menus for User Role Separation

In modern enterprise automation, "platform sprawl" is a significant hurdle. When users are presented with a cluttered interface containing tools they don't need—such as Execution Environment (EE) builders or Git repository management—it increases cognitive load and introduces potential security risks.

The Automation Portal addresses this by providing **RBAC-based navigation menus**, allowing platform architects to tailor the UI experience based on the user's specific role.



Related Pages: This page focuses on controlling which menu items are visible.

Architectural Concept

By tying navigation visibility to RBAC (Role-Based Access Control) groups, administrators can enforce a "least privilege" visual experience. This ensures that:

- **End Users:** Interact only with the automation services they need (Job Templates and History), reducing complexity.
- **Platform Engineers:** Retain full visibility and administrative control over the underlying infrastructure, including EE definitions, content discovery, and synchronization settings.

Reducing UI Complexity through Admin-Defined Access Policies

In large-scale enterprise environments, the Automation Portal can become cluttered with administrative interfaces, repository configurations, and low-level build tools. To improve user experience and maintain security compliance, the Automation Portal allows administrators to define access policies that dynamically tailor the UI based on user roles.



Related Pages: This page focuses on API-level permissions and access control. For information on controlling which menu items are visible to users is based on how the RBAC-based setup is done.

The Concept of UI Minimalism through RBAC

By leveraging Role-Based Access Control (RBAC), you can decouple the complexity of the platform from the needs of the consumer. Reducing UI complexity serves two primary goals:

- **Operational Efficiency:** End users are not overwhelmed by administrative overhead, reducing the cognitive load required to execute automation.
- **Administrative Governance:** By hiding configuration menus (like EE Definitions or Git Repositories), you prevent unauthorized changes to the underlying platform infrastructure.

Architectural Implementation

The portal maps navigation visibility directly to the user's existing Ansible Automation Platform (AAP) permissions. When a user authenticates, the portal checks their associated permissions:

1. **Identity Mapping:** The portal retrieves the user's roles via their API token.
2. **Policy Evaluation:** The system evaluates the user's role against the defined navigation access policies.
3. **UI Projection:** The frontend dynamically renders only the menu items for which the user has authorized access.



The UI policy does not replace server-side security. The system architecture ensures that even if a UI element were exposed, the AI agent and backend services will block any action not permitted by the server-side configuration.

Scenario: Role-Based Navigation Configuration

To implement the requested separation of duties, follow this logical configuration structure:

User Group	Visible Menu Items	Hidden/Restricted Items
End Users	Templates, History	EE Definitions, Collections, Git Repositories, Sync
Platform Engineers	Full Portal (All Items)	None

Configuring the Navigation Policy

To restrict navigation, administrators define access policies within the Portal settings:

1. **Define the Scope:** Identify the target Role (e.g., **cloud-team**) in the AAP RBAC system.
2. **Access Filtering:** Within the Automation Portal administrative settings, navigate to **UI Access Policies**.

3. Map Groups to Menus:

- Create a policy for the `cloud-team`.
- Set the `Allowed Navigation Items` list to strictly include: `['Templates', 'History']`.
- Verify that `System Configuration` and `Builder` menus are excluded.

how to do Step-by-Step Verification

1. **Login as Platform Engineer:** Log in with the `user-admin` account. Verify that the sidebar contains the full suite of options, including **EE Definitions** and **Git Repositories**.
2. **Apply Administrative Filtering:** Navigate to the Portal Admin panel. Select **UI Access Policies** and create a rule for the group containing `user-standard`. Select only **Templates** and **History**.
3. **Login as End User:** Log in with the `user-standard` account. Observe that the navigation sidebar has shrunk, removing all administrative configuration tools.
4. **Verify Backend Protection (Optional):** Attempt to access the URL for the EE Builder directly (e.g., `/portal/ee-builder`). The system should return a 403 Forbidden error, confirming that the UI policy is backed by robust server-side security.

Troubleshooting

- **Cache Issues:** If menu changes do not reflect, clear your browser cache or force-refresh the session.
- **Role Propagation:** If a user still sees extra menus, verify that their user role in AAP was updated and their current session token has been refreshed.

Hands-on Setup: Deploying Automation Portal - Prerequisites and Configuration

In this lab, you'll walk through the essential prerequisite steps to deploy the Ansible Automation Portal on OpenShift Container Platform. We'll configure OAuth authentication with AAP, generate required API tokens, and prepare your OpenShift environment for the portal deployment. These steps ensure secure communication between the portal and your AAP instance.

Overview of What We'll Do

By the end of this lab, you will have:

- Created an OAuth application in AAP for portal authentication
- Generated and secured API credentials
- Authenticated with your OpenShift cluster
- Created a dedicated project for the automation portal
- Prepared all necessary credentials for the Helm chart deployment

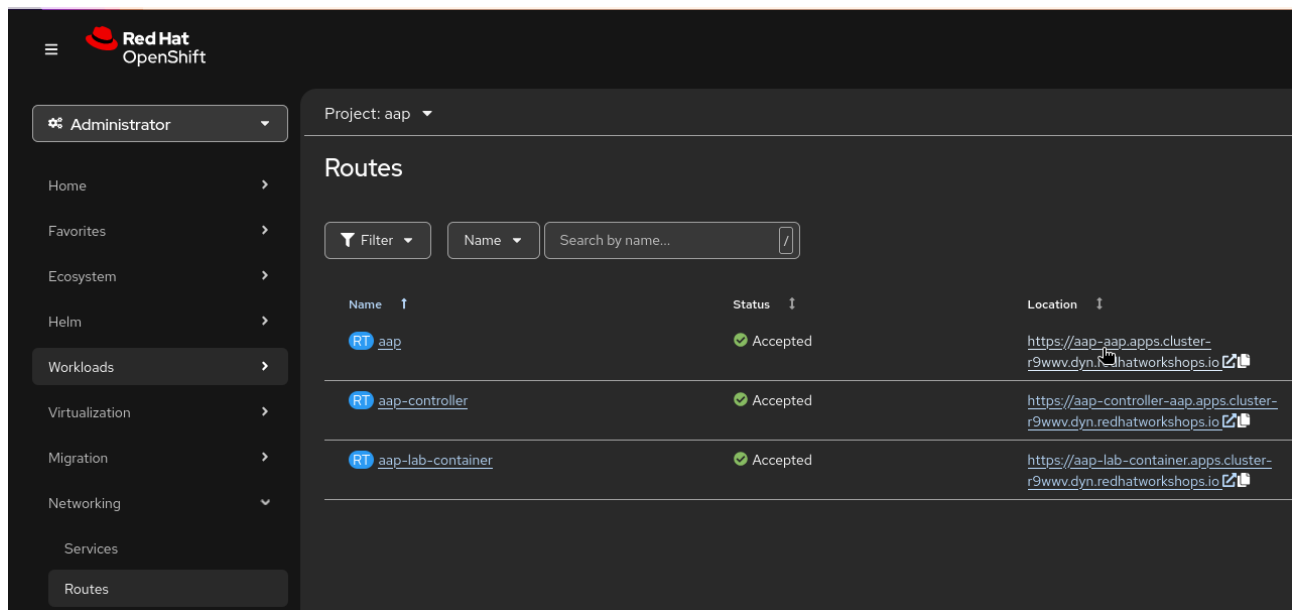
This is the foundational work that must be completed **before** you deploy the Helm chart. Think of it as setting up the "communication channels" between the portal and your AAP instance.

Task 1: Create an OAuth Application in Ansible Automation Platform

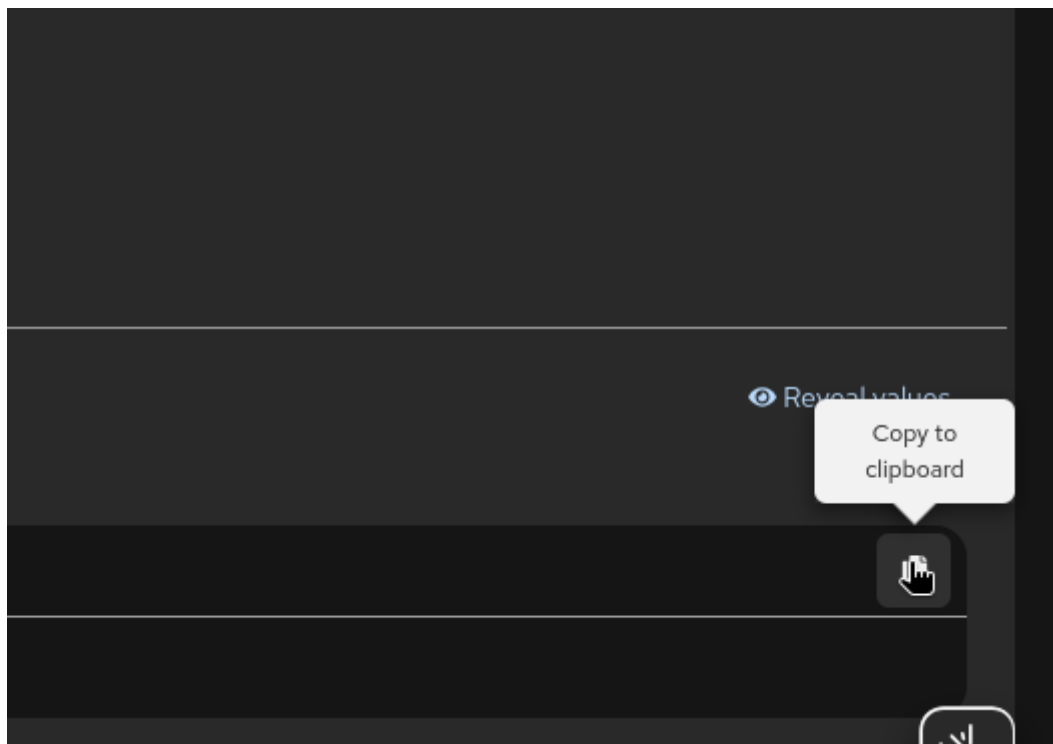
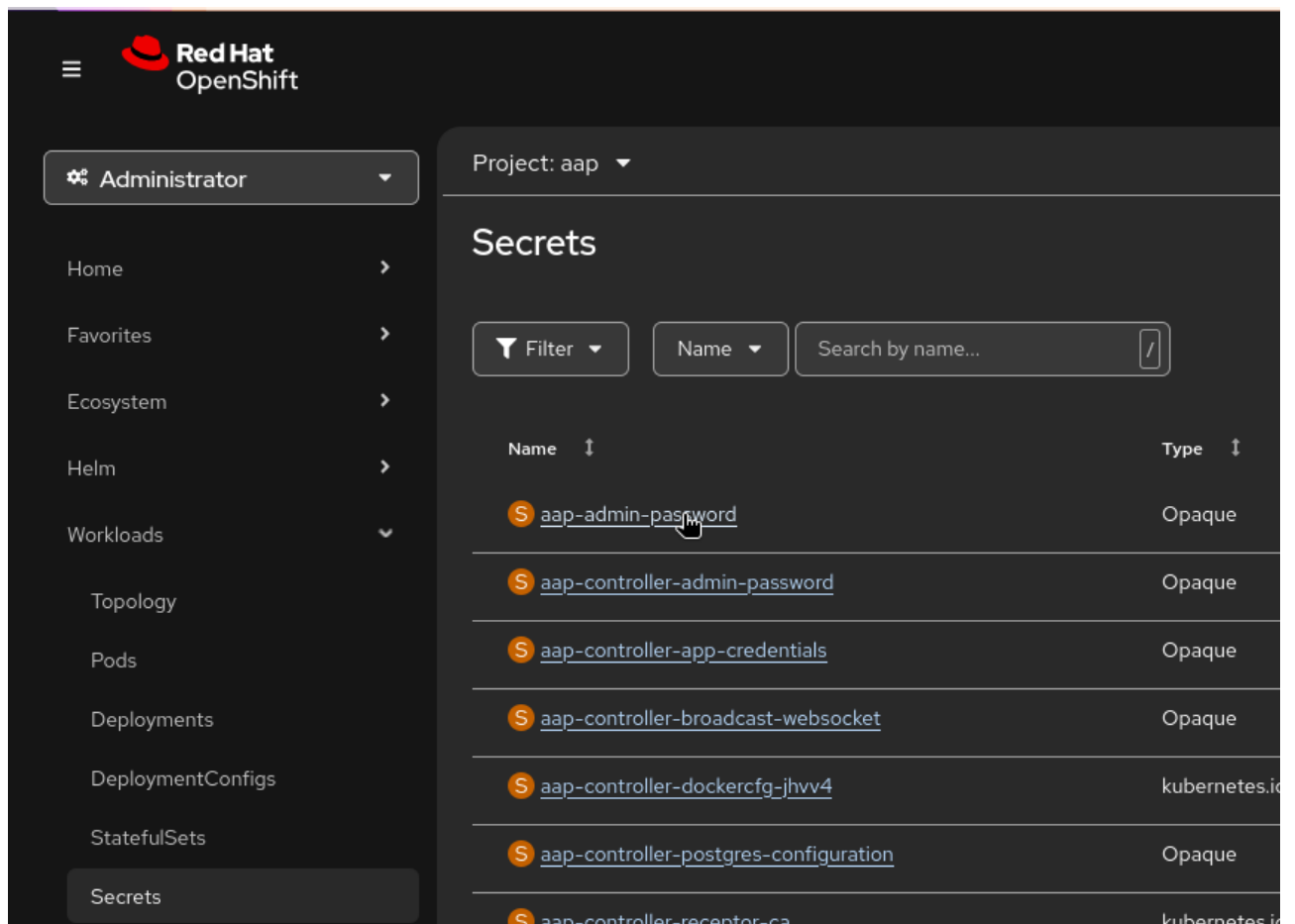
The OAuth application is how the Automation Portal will authenticate users against your AAP instance. Think of it as registering the portal as a "trusted application" within AAP.

Step 1.1: Log in to AAP

1. Login to the Openshift Cluster using {openshift_cluster_console_url} with Username: {openshift_cluster_admin_username} and Password {openshift_cluster_admin_password} and go to Network → Routes to access the Ansible Automation Platform.
2. Select the project as **aap** and open your Ansible Automation Platform instance in a web browser.



3. Go to **Workload → Secrets → aap-admin-password** and use the **administrator** credentials provided to you.



Open your Ansible Automation Platform instance in a web browser using `{aap_controller_web_url}` and log in as user `{controller_username}` with the password you retrieved from the `{controller_password}` secret in the previous step.

```
{aap_controller_web_url}
```

Step 1.2: Navigate to OAuth Applications

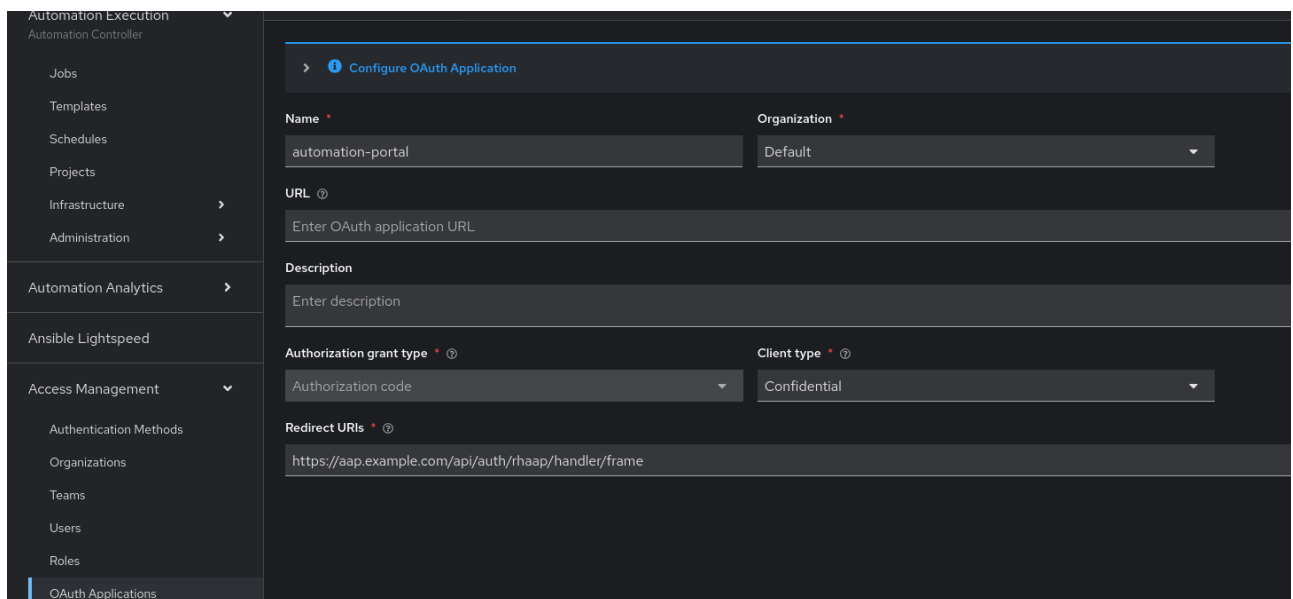
Once logged in:

1. In the left sidebar, click **Access Management**
2. Select **OAuth Applications** from the submenu
3. Click the **Create OAuth Application** button in the upper right corner

Step 1.3: Configure the OAuth Application

Fill in the following fields with these exact values:

- **Name:** `automation-portal`
- **Organization:** Select the organization where your automation content lives (typically "Default" unless you have a multi-tenant setup)
- **Authorization grant type:** Select **Authorization code**
- **Client type:** Select **Confidential**
- **Redirect URIs:** Enter a placeholder URL for now: `https://<automation-portal>/api/auth/rhaap/handler/frame` (we'll update this later with the actual portal deployment URL)



The screenshot shows the 'Configure OAuth Application' form in a dark-themed interface. The left sidebar contains a navigation menu with items like 'Jobs', 'Templates', 'Schedules', 'Projects', 'Infrastructure', 'Administration', 'Automation Analytics', 'Ansible Lightspeed', 'Access Management', 'Authentication Methods', 'Organizations', 'Teams', 'Users', 'Roles', and 'OAuth Applications'. The main form area has the following fields:

- Name:** Text input containing 'automation-portal'.
- Organization:** Dropdown menu showing 'Default'.
- URL:** Text input with placeholder 'Enter OAuth application URL'.
- Description:** Text input with placeholder 'Enter description'.
- Authorization grant type:** Dropdown menu showing 'Authorization code'.
- Client type:** Dropdown menu showing 'Confidential'.
- Redirect URIs:** Text input containing 'https://aap.example.com/api/auth/rhaap/handler/frame'.



The Redirect URI is temporarily set to a placeholder because the actual portal URL won't exist until after you deploy the Helm chart. We'll come back and update this with the real deployment URL in a later step.

Step 1.4: Save and Capture Credentials

Click the **Create OAuth Application** button.

Once created, the system will display:

- **Client ID** — copy this value and save it in a secure location (notepad, password manager, etc.)
- **Client Secret** — this appears only once; copy and save it immediately



IMPORTANT: The Client Secret will never be displayed again. If you lose it, you'll need to regenerate it. Save both values now before navigating away.

Task 2: Enable OAuth Token Creation for External Users

The Below steps will be only needed if you are using an external identity provider (LDAP, SAML, OAuth, etc.) for user authentication. If your users are logging in directly with AAP credentials, you can skip this step for this lab.

1. In the AAP left sidebar, click **Settings**
2. Select **Platform gateway** from the submenu
3. Look for the setting: **Allow external users to create OAuth2 tokens**

If the setting is not already enabled:

1. Click **Edit platform gateway settings**
2. Toggle **Allow external users to create OAuth2 tokens** to **Enabled**
3. Click **Save platform gateway settings**

Verify that the setting now shows as **Enabled** on the Platform gateway settings page.

Task 3: Generate an API Token for the Automation Portal

The API token is used by the Automation Portal to communicate with your AAP instance on a service-to-service basis. This token is what allows the portal to synchronize job templates, manage execution environments, and perform automation tasks.

1. Go to the API 'https://{aap_controller_web_url}/api/gateway/v1/tokens/' and replace the {aap_controller_web_url} with the actual hostname of the Ansible Automation Platform.

```
},
  "user": {
    "id": 3,
    "username": "aap_operator_service_account",
    "first_name": "",
    "last_name": ""
  },
  "created": "2026-08-27T06:20:20.860246Z",
  "created_by": 1,
  "modified": "2026-08-27T06:20:20.895963Z",
  "modified_by": 1,
  "expires": "2027-08-27T06:20:20.836018Z",
  "user": 3,
  "application": null,
  "description": "",
  "last_used": "2026-08-27T09:30:12.018229Z",
  "scope": "write",
  "token": "$encrypted$",
  "refresh_token": null
}
}
```

HTML form Raw data

Application automation-portal
The related application. If None, this is a user token instead of an application token.

Description
A description for this token.

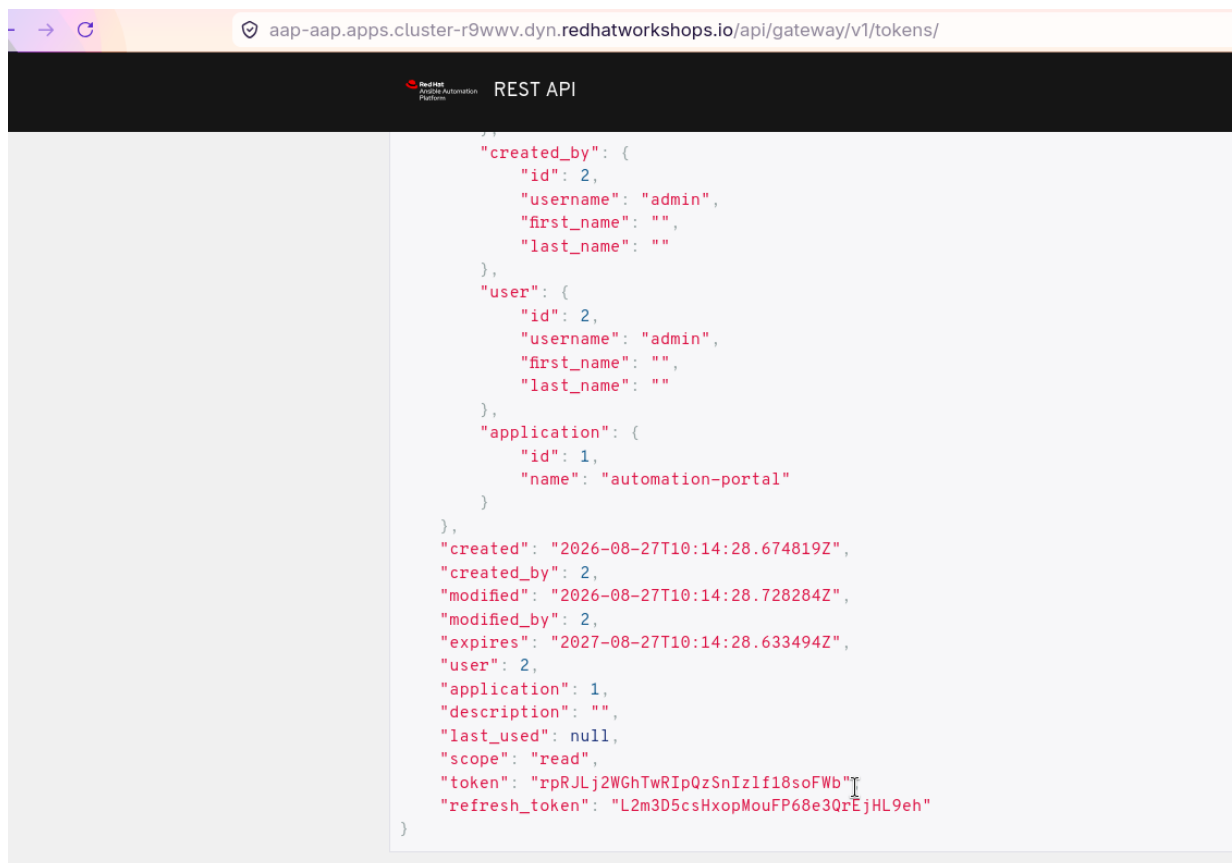
Scope read
Allowed scopes, further restricts user permissions. Must be a simple space-separated string with allowed scopes ['read', 'write']

POST

2. Configure the Token according to the screen shot and POST the Request.

Fill in the following fields:

- **Description:** automation-portal-api-token (descriptive name for your reference)
- **Application:** Select the **automation-portal** OAuth application you created in Task 1
- **Scope:** Select **read** (the portal needs read access to manage templates and execution environments)

A screenshot of a web browser showing a REST API response. The browser's address bar contains the URL 'aap-aap.apps.cluster-r9www.dyn.redhatworkshops.io/api/gateway/v1/tokens/'. The page has a dark header with the 'Red Hat' logo and the text 'REST API'. The main content area is light gray and displays a JSON object. The JSON object has a 'created_by' field with an 'id' of 2 and 'username' of 'admin'. It also has a 'user' field with the same 'id' and 'username'. The 'application' field has an 'id' of 1 and a name of 'automation-portal'. The top-level object includes 'created', 'modified', 'expires', 'user', 'application', 'description', 'last_used', 'scope', 'token', and 'refresh_token' fields. The 'token' field contains a long alphanumeric string.

```
{
  "created_by": {
    "id": 2,
    "username": "admin",
    "first_name": "",
    "last_name": ""
  },
  "user": {
    "id": 2,
    "username": "admin",
    "first_name": "",
    "last_name": ""
  },
  "application": {
    "id": 1,
    "name": "automation-portal"
  }
},
"created": "2026-08-27T10:14:28.674819Z",
"created_by": 2,
"modified": "2026-08-27T10:14:28.728284Z",
"modified_by": 2,
"expires": "2027-08-27T10:14:28.633494Z",
"user": 2,
"application": 1,
"description": "",
"last_used": null,
"scope": "read",
"token": "rpRjLj2WghTwRipQzSnIzlf18soFWb",
"refresh_token": "L2m3D5csHxopMouFP68e3QrEjHL9eh"
}
```

Generate and Save the Token and as the system will display the generated token — copy it immediately and save it in a secure location alongside your Client ID and Client Secret from Task 1.



IMPORTANT: Like the Client Secret, the API token is displayed only once. If you navigate away, you cannot retrieve it again without creating a new token.

Make a note of the three values you now have:

- Client ID: _
- Client Secret: _
- API Token: _

These three values are critical for the next phase of setup.

Task 4: Authenticate with Your OpenShift Cluster

Now we shift focus to the OpenShift side of the setup. We need to connect to your OCP cluster using the CLI tool, which will be the gateway for creating the portal's project and secrets.

1. Login to the bastion host using the credentials from the RHDP login page.
2. Once login to bastion host then login to OCP cli using the command shared in **OpenShift CLI Commands** section on the RHDP login page:

```
oc login -u admin -p {openshift_cluster_admin_password} {openshift_api_server_url}
```

```
[lab-user@bastion ~]$ oc login -u admin -p QpKQDdbdvd91
https://api.cluster-r9wwv.dyn.redhatworkshops.io:6443
Login successful.

You have access to 83 projects, the list has been suppressed. You can list all projects with 'oc projects'

Using project "default".
```

3. Run the following command, replacing **portal** with your preferred project name if you want a different project name(keep it lowercase with hyphens only, no underscores):

```
oc new-project portal
```

4. Make sure your CLI context is set to the newly created project:

```
oc project portal
```

5. Build and Deploy the Plugin Registry
6. Go to the [Installer](#) link and Get the installer link for **Ansible automation portal Setup Bundle**.

Product Software

Packages

Errata

Installers and Images for Red Hat Ansible Automation Platform (v. 2.7 for RHEL 10 for x86_64)

Ansible Automation Platform 2.7 Containerized Setup Last modified: 2026-08-24 SHA-256 Checksum: 4ce218df955e57e7f7a6a3365ac59f819616fa33b49046a4cac4905bdf02075b	Download Now 11 MB
Ansible Automation Platform 2.7 Containerized Setup Bundle Last modified: 2026-08-26 SHA-256 Checksum: 20eec4a012c1864bbc77c83f9abd23c600958a81ead7292d3a4e201e57ab7336	Download Now 3.76 GB
Ansible automation portal Setup Bundle Last modified: 2026-08-18 SHA-256 Checksum: 38135ed58362b43791add54f72428edae0c0f21e3	Download Now 3 MB
Ansible automation portal RHEL Installer (QCOW2) Last modified: 2026-08-06 SHA-256 Checksum: c21022530b6f1a54642571c83928ed29dc078d294	Download Now 16 GB
Ansible automation portal RHEL Installer	Download Now

self-service-automation-portal-plugins-2.2.6.tar.gz

Open Link in New Tab

Open Link in Split View (M)

Open Link in New Window

Open Link in New Private Window

Preview Link (J)

Bookmark Link...

Save Link As...

Copy Link

Copy Clean Link (U)

7. Download the installer file to the VM:

```
wget -O automation.tar.gz "<paste-link-here taken from above 5>"
```

8. Extract the **automation.tar.gz** file using the following command:

```
tar -xvf automation.tar.gz
```

- Run the following commands to create the **plugin-registry** binary build, build the image using the dynamic plugin directory, and deploy the resulting image:

Where **\$DYNAMIC_PLUGIN_ROOT_DIR** is the local directory containing the dynamic plugin files.

```
export DYNAMIC_PLUGIN_ROOT_DIR=/home/lab-user
```

```
oc new-build httpd --name=plugin-registry --binary
```

```
oc start-build plugin-registry --from-dir=$DYNAMIC_PLUGIN_ROOT_DIR --wait
```

```
oc new-app --image-stream=plugin-registry
```

- b. Go to OpenShift Web Console, navigate to the **portal** project, and Then Workloads → Topology verify that the **plugin-registry** pod is running successfully.

Create OpenShift Secrets for AAP Authentication

OpenShift Secrets are secure, encrypted storage for sensitive data. We'll create a secret that holds the OAuth credentials and API token you generated in Tasks 1 and 3. The Automation Portal Helm chart will reference this secret during deployment.

1. Go to Workloads > Secrets in the OpenShift console and make sure the project **portal** is selected. You should create the secret **secrets-rhaap-portal** with the below information. You can also create the secret using the **oc** command line tool.

Create key/value secret



Key/value secrets let you inject sensitive data into your application as files or environment variables.

Secret name *

secrets-rhaap-portal

Unique name of the new secret.

⊖ Remove key/value

Key *

aap-host-url

Value

Drag a file here or browse to upload

Browse...

Clear

https://aap-aap.apps.cluster-r9www.dyn.redhatworkshops.io

Drag and drop file with your value here or browse to upload it.

⊖ Remove key/value

Key *

oauth-client-id

Value

Drag a file here or browse to upload

Browse...

Clear

BLnmehRz9Bjw7Wuy4r08cvl5a8tAXGU8Yd0Hnlpn

Drag and drop file with your value here or browse to upload it.

Project: portal ▾

Drag and drop file with your value here or browse to upload it.

Remove key/value

Key *

oauth-client-secret

Value

Drag a file here or browse to upload

Browse...

Clear

tZqzSXO1bFihOPV1u6yPthe5UQKvAbYLPi8CbUu0iiFK36KjOaxrYwFA5dle9hSA6ryO3cbS1Lh97hvFSJ9WGs2D1Rc3yHfswNCoGYoTIRhGKgSITHb1wUgDIWydrLbr

Drag and drop file with your value here or browse to upload it.

Remove key/value

Key *

aap-token

Value

Drag a file here or browse to upload

Browse...

Clear

rpRJLj2WGhTwRIpQzSnIzlfI8soFWb|

Drag and drop file with your value here or browse to upload it.

Add key/value

Create

Cancel

OR

You can also run the secret creation command, gather the values from Tasks 1 and 3:

19

- `<aap-instance-url>` — The URL of your AAP instance (e.g., `https://{aap_controller_web_url}`)
- `<oauth-client-id>` — The Client ID from the previous task.
- `<oauth-client-secret>` — The Client Secret from the previous task.
- `<aap-token>` — The API Token from the previous task.

1. Run the following command, replacing the placeholders with your actual values:

```
oc create secret generic secrets-rhaap-portal \
  --from-literal=aap-host-url="<your-aap-url>" \
  --from-literal=oauth-client-id="your-client-id-here" \
  --from-literal=oauth-client-secret="your-client-secret-here" \
  --from-literal=aap-token="your-api-token-here" \
  -n portal
```



The secret name (`secrets-rhaap-portal`) is important—the Helm chart expects this exact name. If you change it, you'll need to update your Helm values during deployment.

Verify the Secret Was Created

Confirm the secret exists:

```
oc get secret secrets-rhaap-portal -n portal
```

You should see output like:

NAME	TYPE	DATA	AGE
secrets-rhaap-portal	Opaque	4	10s

Once the above is done. You're now ready for the next phase: deploying the Automation Portal via Helm chart!

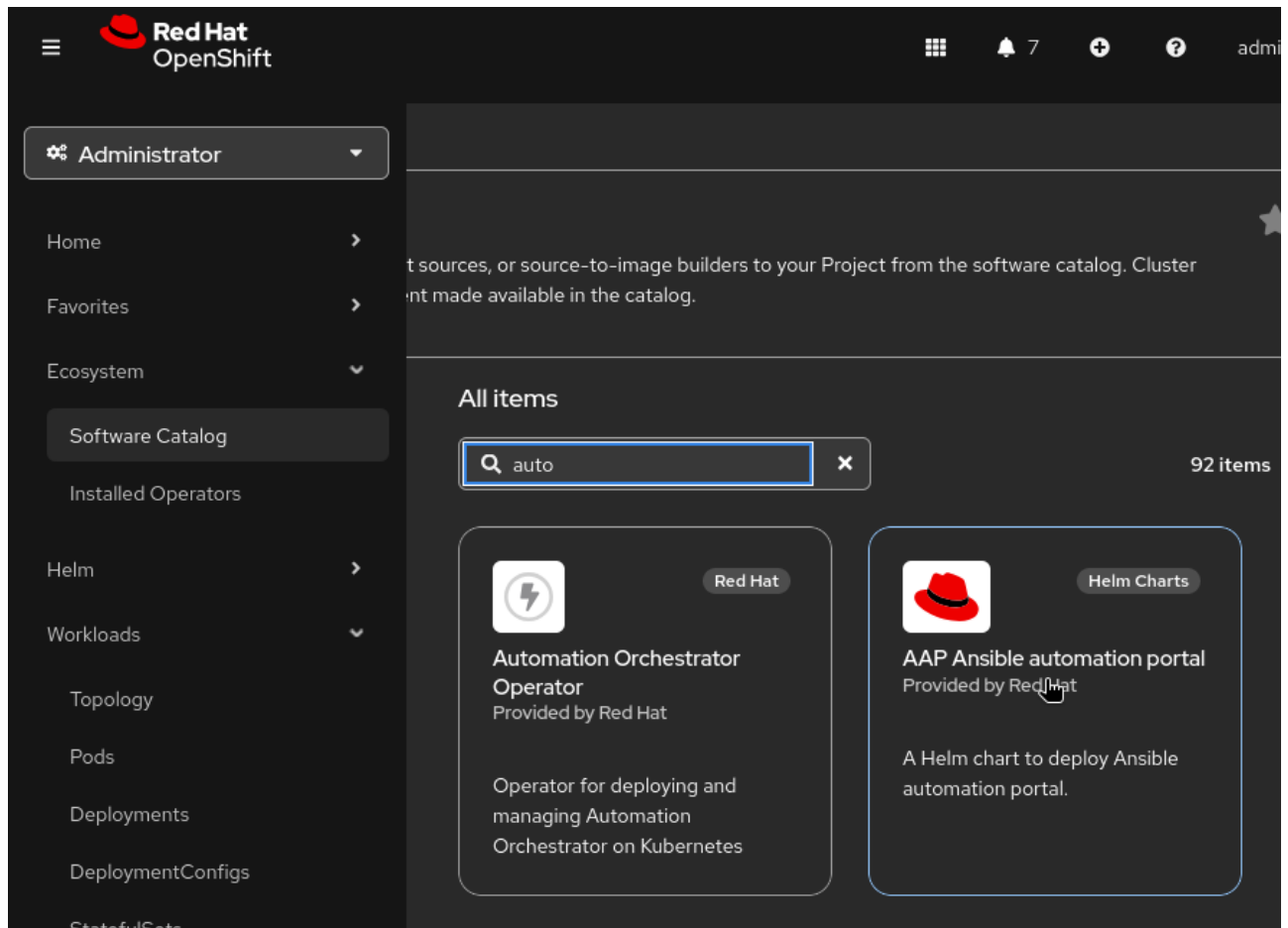
Hands-on Lab: Installing and Verifying the Automation Portal on OCP

In this lab, you will deploy the Automation Portal on your OpenShift Container Platform (OCP) environment.

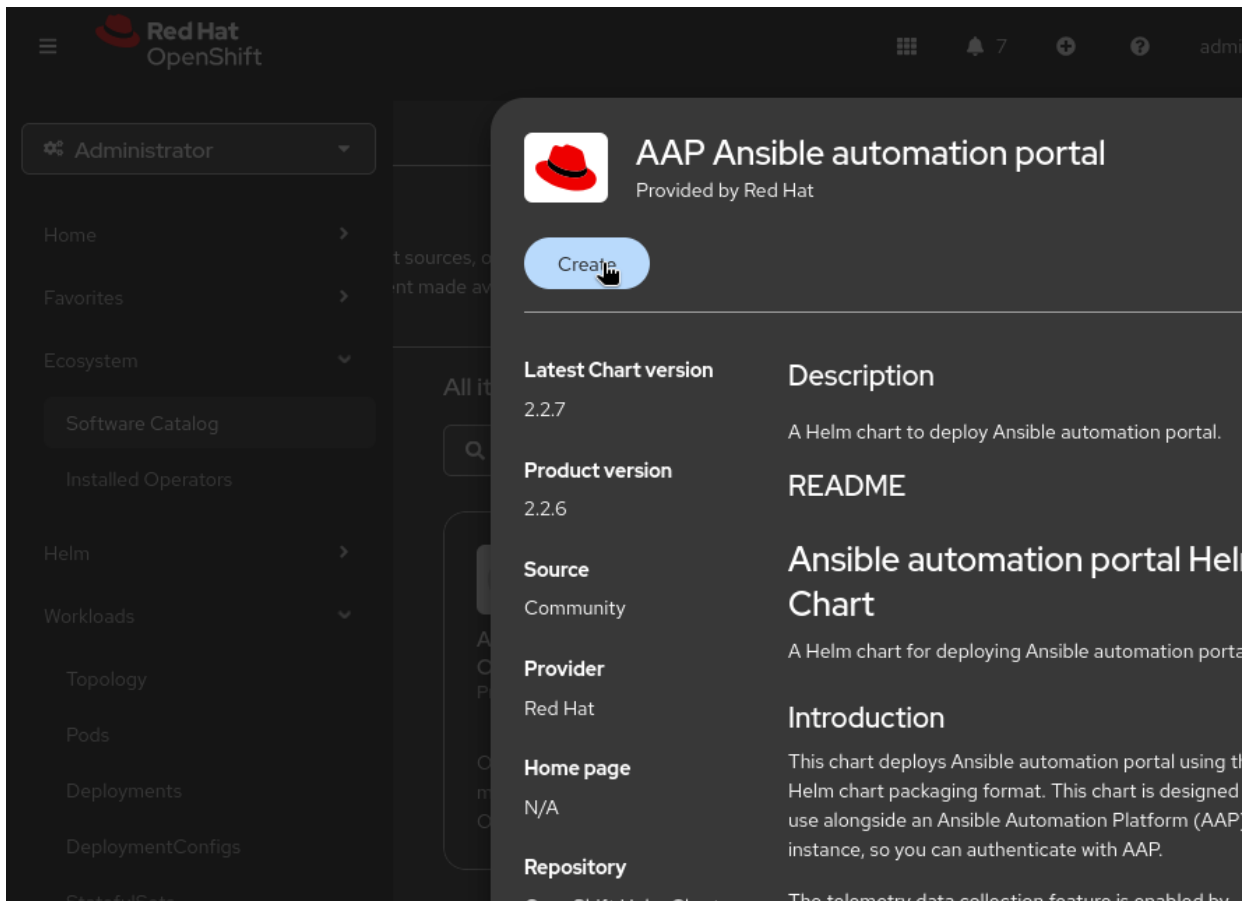
Installing the Automation Portal

1. Login to the Openshift Cluster using `{openshift_cluster_console_url}` with Username: `{openshift_cluster_admin_username}` and Password `{openshift_cluster_admin_password}` and go to Network → Routes to access the Ansible Automation Platform.

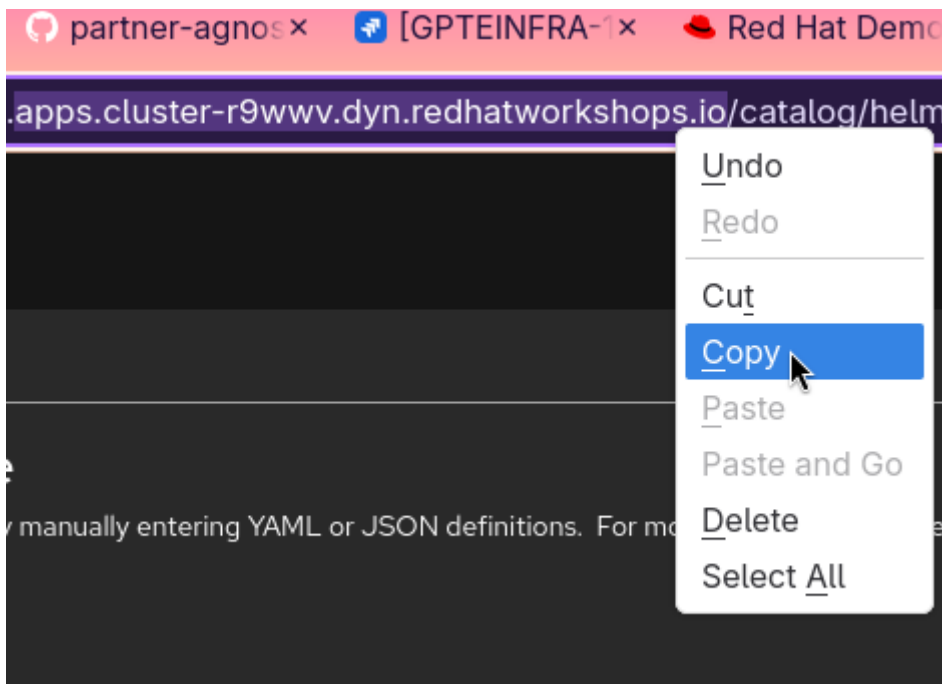
2. Go to Ecosystem > Software Catalog in the OpenShift web console.
3. Search for "Automation Portal" in the catalog and select it.



- a. Click on **Create** to start the installation process.



- b. Change the Release Name to `automation-portal` and you can change it if you want.
- c. Change the `clusterRouterBase` to the correct value for your cluster. For example, if the OCP cluster url is `https://console-openshift-console.apps.cluster-kstzt.dyn.redhatworkshops.io` then the `clusterRouterBase` should be `apps.cluster-kstzt.dyn.redhatworkshops.io`.



Project: portal ▾

Create Helm Release

The Helm Release can be created by manually entering YAML or JSON definitions. For more information on the chart, refer to the chart's documentation.

Release name *

automation-portal

A unique name for the Helm Release.

Configure via: ☐ Form view ☒ YAML view

🔔 Form view is disabled for this chart because the schema is not available

```

1 redhat-developer-hub:
2   global:
3     _environment:
4       _development: false
5       _production: true
6       _stage: false
7   clusterRouterBase: apps.cluster-r9wwv.dyn.redhatworkshops.io
8   dynamic:
9     plugins:

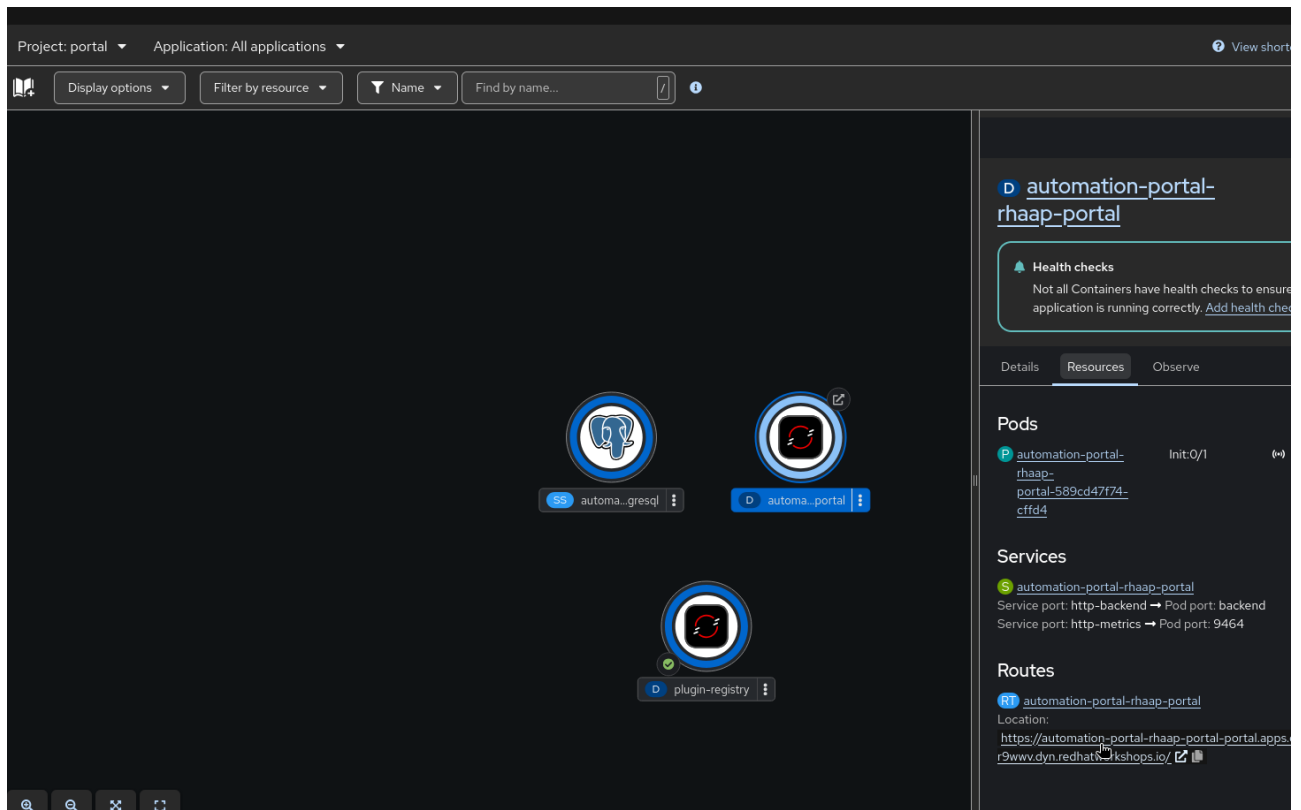
```

- d. By default, it will be setting up at the Organization **Default** for the automaton portal the parameter is orgs if you wanted to change it.

Once the above settings are done, click on **Create** to start the installation process.

The installation process will take a few minutes. You can monitor the progress Workloads > Topology in the portal project. Once the installation is complete, you will see the Automation Portal components running.

- Go to Workloads > Topology in the portal project. Once the installation is complete, you will see the Automation Portal components running.
- Click on **automation-portal-rhaap-portal** resource to view the details. You will find the route URL in the **Routes** section. This URL is used to access the Automation Portal web interface.



6. Copy the route URL and go to Ansible Automation Platform > Access Management > OAuth Applications.
7. Edit the automation-portal OAuth application to include the route URL as a valid redirect URI and save the changes. This step is crucial for the OAuth authentication to work correctly.

```
https://<aap-portal-hostname>/api/auth/rhaap/handler/frame
```

OAuth Applications > Edit automation-portal

Edit automation-portal

> [Configure OAuth Application](#)

Name * automation-portal Organization * Default

URL ? Enter OAuth application URL

Description Enter description

Authorization grant type * ? Authorization code Client type * ? Confidential

Redirect URIs * ? <https://automation-portal-rhaap-portal-portal.apps.cluster-r9www.dyn.redhatworkshops.io/api/auth/rhaap/handler/frame>

Save OAuth application Cancel

8. Access the Web UI of the Automation portal and click on the **Login with Ansible Automation Platform** button. You will be redirected to the AAP login page. Enter your AAP credentials to log in.

← → ↺ automation-portal-rhaap-portal-portal.apps.cluster-r9www.dyn.redhatworkshops.io

Ansible automation portal

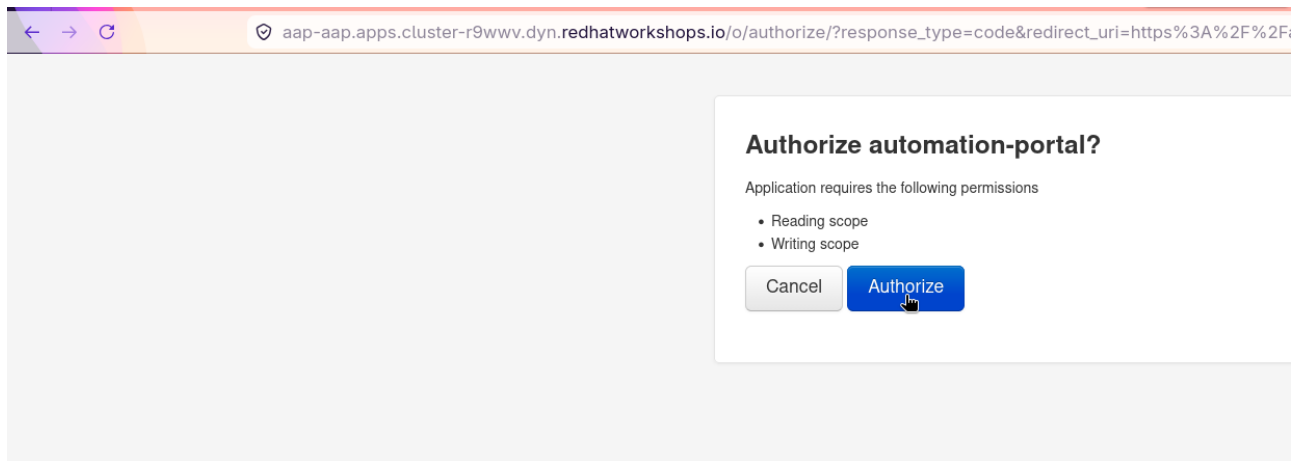
Select a Sign-in method

Ansible Automation Platform

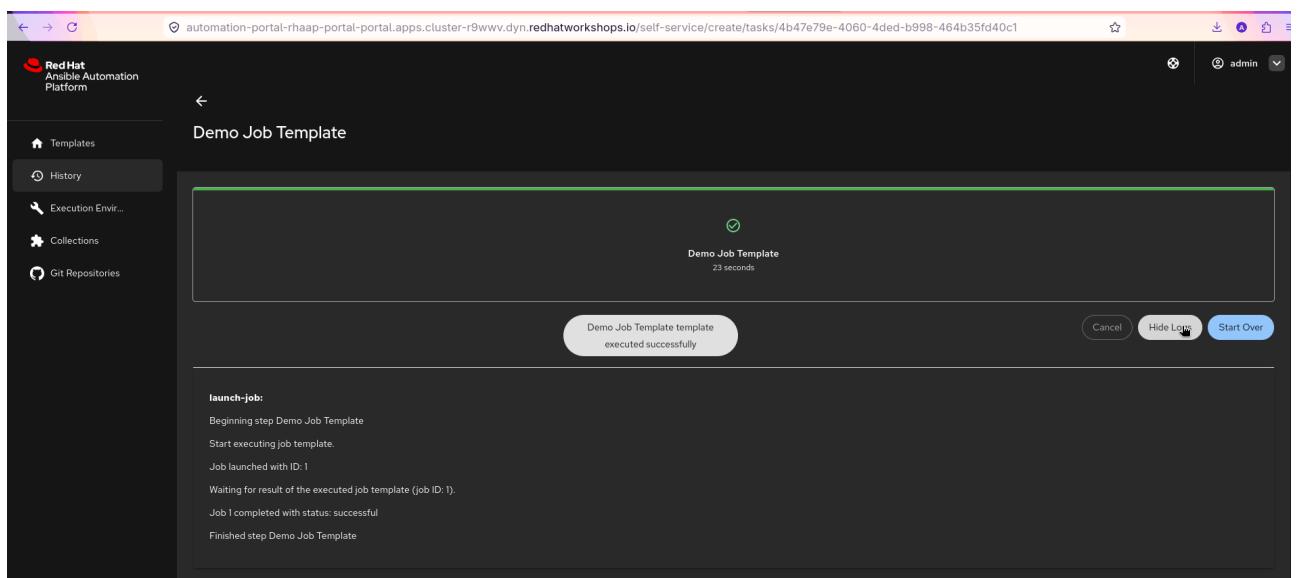
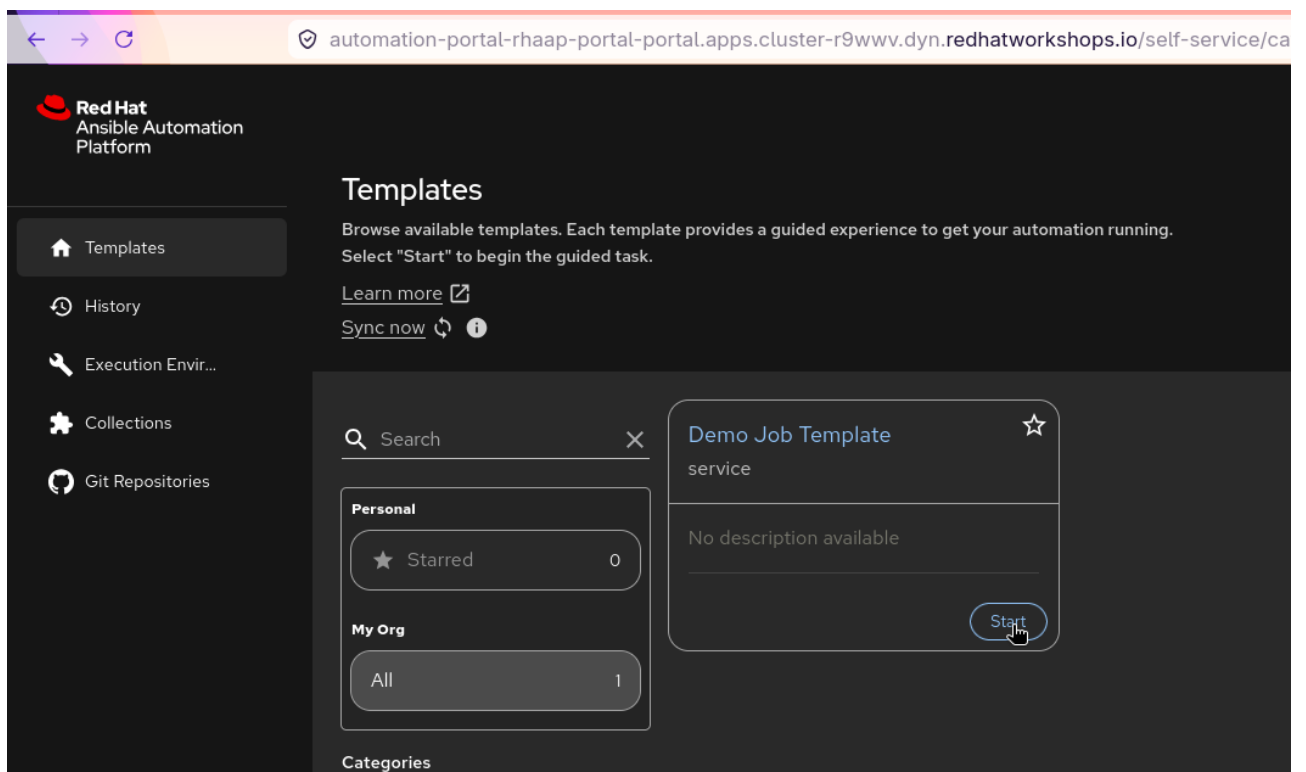
Sign in using Ansible Automation Platform

Sign in

9. Once logged in, you need to provide the necessary permissions. click authorize and you will see the Automation Portal dashboard.



10. Click on the Demo Job Template and click on **Launch** to execute the job. You will see the job execution status and output.



This is All about setting up the Automation Portal on OCP and verifying its functionality. You can now explore the features of the Automation Portal like RBAC Control and the execution of job templates and workflows via specific users.

For more Information on the Automation Portal or for RBAC hands on lab, refer to the [Introduction to AAP Self-Service Automation](#)

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)